

Christmas Decor Inventory Web App — Feature & Function Research

Prepared for: John, Christmas Decor franchise owner **Use case:** Vibe-coded inventory web app integrated with Jobber, managing rented holiday lighting products **Source basis:** Jobber Developer Center, Christmas Decor business model, leading rental/inventory platforms (Booqable, Rentman, EZRentOut, Goodshuffle Pro, Current RMS, HireHop, Point of Rental, Texada, Quiply, RealGreen, QuotelQ, Holiday Home Concepts)

1. Executive Summary

Your business model is **rental-with-storage-per-client**, which is genuinely different from typical equipment rental. Traditional rental software assumes pool-based inventory (any unit fulfills any order). Christmas Decor's model is closer to "**customer-assigned kits in long-term cycle**": a client's display gets designed once, the same components return to the same client most years, items are stored year-round at your facility, and the asset lifecycle includes a refurbishment phase between seasons.

This means your inventory system has to handle **three overlapping concepts** that most off-the-shelf rental apps blur together:

1. **Master inventory** — every physical item you own (strands, garlands, wreaths, controllers, timers, stakes, cords, C9s, mini-lights, animated displays).
2. **Customer kits** — the specific bin(s) of items assigned to a client's property, pre-built and stored.
3. **Job allocations** — what's currently checked out for an active install, what's scheduled to come back, what's being repaired. This layer also tracks:
 - **What needs to be replaced** — at takedown or during inspection, items flagged as damaged/worn/failed are queued for replacement, either from spare stock or via a purchase order, before next season's deployment.
 - **Auto-reserve on job sold** — the moment a Jobber Quote is converted to a Job (or moves to a "sold/won" state), the kit's components are immediately marked as reserved and pulled from available stock. This prevents the same physical items from being promised to two clients during the busy quoting window.

The Jobber integration is the spine that ties this to your existing CRM/scheduling/invoicing, so you avoid double entry.

2. Christmas Decor Business Context (How It Shapes the App)

From the Christmas Decor service model, your operational cycle is:

- **Design consultation** → quoting, line-item buildout
- **Complete installation** → kit pulled from storage, dispatched to job site
- **Proactive maintenance** → mid-season service visits
- **Timely takedown** → return to facility
- **Year-round storage** → inventory locked to client until next season

Implications for the app:

- Items have a **"home bin"** (long-term storage location) AND a **"current location"** (facility / in-transit / at-property / in-repair).
 - Most clients are **renewing**, not new — the same kit gets pulled, refreshed, redeployed.
 - **New customer onboarding workflow** — when a brand-new client is sold, the app needs a guided flow to spec and build their first kit from master inventory: design line items → reserve components → assign a new bin → assemble → label → store. This is distinct from the renewing-customer flow because nothing pre-exists for them.
 - **Vehicle inventory** — your trucks and trailers are themselves trackable assets, AND they act as mobile locations during install/takedown days. The app needs to: (a) maintain a list of vehicles (make, model, plate, capacity, registration, insurance); (b) treat each vehicle as a `location` so items can be scanned "into" and "out of" a specific truck; (c) optionally track on-vehicle "rolling stock" (extension cords, ladders, zip ties, tools, spare bulbs) that lives on the truck all season.
 - **Condition tracking** matters more than in normal rental: bulbs burn out, garlands fray, controllers fail, and you need a record per item per season.
 - **Pre-season build season** (summer/fall) is when crews assemble new kits or refurbish existing ones — the app needs a "build mode" workflow.
 - Multi-segment customers: **residential, commercial, municipal** — each has different product mix, contract terms, and reporting needs.
-

3. Jobber API — What You Get For Free

Jobber's API is GraphQL-based. Key objects and how to use them:

Objects you'll read/write

- **Client** — your customer. Has billing address, properties, custom fields.
- **Property** — physical address where work is performed. Multiple per client.
- **Request** — initial inquiry from a lead.
- **Quote** — line-item estimate. Has `lineItems` and `quoteStatus`.
- **Job** — the scheduled work. Has `lineItems`, `visits`, `jobStatus`, `jobNumber`.
- **Visit** — a single scheduled trip (install, maintenance, takedown). Has `assignedUsers`, `startAt`, `endAt`, `isComplete`.
- **Invoice** — billing record.
- **ProductOrService** — Jobber's product catalog. **NOTE: This is a basic catalog, not an inventory system.** It has `name`, `defaultUnitCost`, `category`, `description` — no quantities, no serial numbers, no location. This is exactly why you're building your own app.
- **User** — Jobber team members.
- **Custom Fields** — your app can configure up to **5 custom fields per Jobber object type**. Useful for stamping a Jobber Job with the inventory `kit_id`, or stamping a Client with a `customer_kit_id` link back into your system.

Webhooks (real-time sync)

You can subscribe to events like `CLIENT_CREATE`, `CLIENT_UPDATE`, `JOB_CREATE`, `JOB_UPDATE`, `QUOTE_CREATE`, `INVOICE_CREATE`, `VISIT_COMPLETE`, etc. Jobber sends an HMAC-signed POST to your URL. Important constraints:

- You must respond **within 1 second** — process asynchronously (queue it).
- Delivery is **at-least-once** — your handler must be idempotent (deduplicate by event ID).
- You verify authenticity with the `X-Jobber-Hmac-SHA256` header using your OAuth client secret.

What Jobber doesn't do (and your app must)

- Track individual physical units (serials, barcodes).
- Real-time stock counts.
- Bin/location tracking.
- Kit/bundle assembly with component availability roll-ups.
- Maintenance history per item.
- Reservation conflict detection.
- Condition grading and depreciation.

Auth model

OAuth 2.0. You'll register an app in the Jobber Developer Center, get a client ID/secret, request scopes (read/write clients, read/write jobs, custom_fields, etc.), and redirect users through the OAuth flow once.

4. Comprehensive Feature List

Organized by functional area. Items marked **[CORE]** are must-haves for v1; **[V2]** can come later; **[INDUSTRY-SPECIFIC]** is unique to your Christmas Decor model.

4.1 Inventory Item Management **[CORE]**

- Master item catalog with categories — at minimum: **Holiday/Seasonal Lighting** (C9 strands, mini lights, garlands, wreaths, mega trees, wire frames, animated displays), **Permanent Lighting** (year-round outdoor fixtures, cloud-controlled systems like the 365-style products), **Landscape Lighting** (path lights, spotlights, transformers, low-voltage cabling), **Controllers & Electrical** (controllers, timers, smart hubs, extension cords, splitters), **Hardware & Install Materials** (clips, stakes, zip ties, fasteners), **Equipment** (ladders, lifts, tools), **Vehicles** (trucks, trailers).
- Item attributes: SKU, name, description, manufacturer, model, color/finish, length/size, voltage/wattage, photos.
- **Per-unit serial tracking** — each physical strand or controller gets a unique ID/barcode/QR (best-in-class apps do RFID at scale).
- **Bulk vs. serialized items** — some items (e.g., a 100-ft strand) are serialized; consumables (zip ties, replacement bulbs) are tracked by quantity.
- **Variants** — same product, different sizes/colors as variants of one parent.

- **Cost & pricing fields** — purchase cost, replacement cost, rental rate, lifetime hours.
- **Lifetime tracking** — purchase date, total seasons deployed, total hours lit, retirement date.
- **Per-item retirement threshold** — each item carries its own configurable lifespan limit (e.g., max seasons before mandatory retirement, max repair count, condition floor). Different products have very different lifespans: a C9 strand might retire after 4–5 seasons, a wire frame after 10+, a controller when it fails. Set a default at the **category level**, override at the **item level** when a specific unit needs different treatment. The system auto-flags items approaching their threshold for inspection before next season.
- **Condition grading** — A/B/C/Retired, with notes and photos per inspection.
- **Custom fields** — admin-configurable so you can add attributes without a code change.
- **Tags / labels** — flexible cross-cutting categorization (e.g., “Warm White”, “Animated”, “Commercial Grade”).

4.2 Asset / Kit / Bundle Tracking [CORE + INDUSTRY-SPECIFIC]

This is the biggest gap between Jobber and what you need.

- **Nested kits** (kits-within-kits) — a “Front Roofline Package” kit might contain three “75ft C9 Strand” kits plus one controller kit. Best-in-class examples: Current RMS, Rentman, HireHop.
- **Customer-assigned kits** [INDUSTRY-SPECIFIC] — bind a kit to a `client_id / property_id` so it follows the client year-over-year.
- **Kit build workflow** — “build mode” where a crew pulls components from master inventory and assembles a kit, scanning each item in.
- **Real-time component availability roll-up** — if a kit needs 5 strands and only 3 are available, the system flags conflict.
- **Substitution rules** — if a specific 50ft warm-white strand is unavailable, allow auto-suggestion of equivalent (Goodshuffle and Current RMS both do this).
- **Kit templates** — reusable blueprints (e.g., “Standard 2,500 sq ft Roofline Package”) that get instantiated per client.
- **Bin / location hierarchy** — Warehouse → Aisle → Rack → Bin → Item. Crew needs to know “go to Aisle 3, Bin 47” to pull the Smith family kit.
- **QR / barcode labels** for both bins and items.

4.3 Customer & Property Sync (Jobber Integration) [CORE]

- **One-time OAuth connection** to Jobber.
- **Two-way sync** of clients and properties:
 - Webhook-driven inbound: when a client/property is created or updated in Jobber, mirror to your DB.
 - On-demand outbound: write back custom fields to Jobber (e.g., "Kit ID", "Bin Location", "Last Inspection Date") so your office staff sees them in Jobber's UI without leaving Jobber.
- **Property → Kit linkage** — every Jobber Property can have one or more kits associated.
- **Job → Kit linkage** — when a Jobber Job is created, your app surfaces "this client's kit is in Bin 47" automatically.
- **Visit → Allocation linkage** — when a Visit happens (install, maintenance, takedown), the app records which items were checked out/in.
- **Reverse-search** — given a bin or item, find which Jobber client owns it.
- **Reconciliation report** — items in Jobber Products that aren't in inventory, and vice versa.

4.4 Reservations, Allocation & Scheduling [CORE]

- **Availability calendar** — visual Gantt or calendar showing what's allocated when.
- **Conflict detection** — prevent double-booking the same physical item.
- **Reserved / allocated / checked-out / returned states** for every line item.
- **Drag-and-drop scheduling** (Texada, Goodshuffle, Rentman all do this).
- **Pull list / pick list generation** — the night before an install, generate a printable/scannable list of what every crew needs to grab.
- **Auto-generation of pull lists from Jobber Visits** — when a Visit is scheduled, the kit's bill-of-materials becomes the pull list.
- **Buffer/grace periods** — install days and takedown days don't overlap exactly with billing periods.
- **Recurring annual booking** [INDUSTRY-SPECIFIC] — auto-roll a client's previous-season kit into next season's reservation as a draft.

4.5 Mobile / Field Operations [CORE]

- **Mobile-first responsive design** OR a true mobile app — your install crews live on their phones.
- **Barcode/QR scanning via phone camera** — check items out to a vehicle, check in at takedown.
- **Bulk scan mode** — scan 30 items in 30 seconds at a loading dock.
- **Photos in the field** — installers take before/after photos that attach to the job and the kit. RealGreen specifically calls this out for holiday lighting.
- **Condition flagging on takedown** — installer marks “C9 strand #4421 — 3 burned bulbs, broken socket on end” as the item comes off the truck.
- **Offline support** — crews are sometimes in spotty signal areas; queue scans and sync when reconnected.
- **Geo-stamping** of scan events — verify the item was scanned at the property, not at the warehouse.
- **Today’s job view** — pulled from Jobber Visits, shows what’s happening today, what kits go where, what bins to pull from.
- **Digital signatures** — capture client signoff at install completion.

4.6 Vehicle Inventory & Rolling Stock [INDUSTRY-SPECIFIC]

Trucks and trailers are dual-purpose: they’re tracked assets AND they’re mobile locations.

- **Vehicle registry** — make, model, year, license plate, VIN, capacity (cubic ft / weight), insurance expiry, registration expiry, fuel type, odometer.
- **Vehicle as location** — each vehicle gets a `location_id` so items scanned onto it show up “in transit” or “loaded for Smith install.”
- **Rolling stock** — items that live permanently on a vehicle through the season (extension cords, ladders, hand tools, spare bulbs, zip ties, fuses, replacement clips). Track these separately from job-specific kit items so you don’t double-count.
- **Daily load manifest** — at start of day, generate the load list for each truck based on the day’s Jobber Visits + each client’s kit. Items are scanned onto the truck; system flags missing items.
- **End-of-day reconciliation** — at return, scan everything off the truck. Anything that didn’t come back is auto-flagged (left at property, lost, in repair).

- **Truck assignment to a job** — each Jobber Visit can be tagged with which truck is fulfilling it, so the load manifest knows which vehicle to load and the post-job reconciliation knows where items should land.
- **Vehicle maintenance log** — oil changes, tire rotations, registration renewals, accident records. Tie into the vehicle policy you've already developed.

4.7 Maintenance & Lifecycle Management [INDUSTRY-SPECIFIC]

- **Maintenance ticket queue** — items flagged at takedown auto-create a repair ticket.
- **Repair workflow** — assign a technician, log parts used, log labor time, mark as repaired or retired.
- **Refurbishment season tracking** — "X% of inventory has been inspected/refurbished and is ready for next season."
- **Bulb-out / failure tracking** with statistics over time per product line (which manufacturers fail most).
- **Preventive maintenance schedules** — automated reminders based on age or hours-of-use.
- **Retirement / disposal log** — items removed from rotation with reason, date, salvage value.
- **Warranty tracking** — date purchased, warranty expiration, RMA status with vendor.

4.8 User Roles & Permissions [CORE]

You specified Admin / Manager / User. Suggested permission matrix:

Admin

- Full read/write everything.
- Manage other users (create, deactivate, change roles).
- Configure system settings, custom fields, categories.
- Manage Jobber connection / OAuth tokens.
- Access financial and cost data.
- Bulk import/export.
- View audit logs.

Manager

- Read/write inventory, kits, allocations.
- Approve maintenance tickets.
- Generate reports.
- Cannot delete users or change global settings.
- Cannot see unit cost / margin data (configurable).
- Can override allocations and resolve conflicts.

User (warehouse/ops staff who actively use the app — NOT field installers)

- Read-only on most inventory.
- Scan items in/out for vehicles and bins.
- Submit condition reports and photos.
- View open jobs from Jobber.
- Cannot see pricing or financial data.
- Cannot edit master records.

Note on field crew: Field installers and takedown crew are **not users of this app**.

Inventory is tied to vehicles and warehouse bins, not to people. The User role exists for the smaller group of warehouse/ops staff who actively log in to scan, load trucks, and process returns. Field crew operate physically; their actions are recorded against the vehicle they're using, not against their personal account.

Additional considerations:

- **Field-level permissions** for sensitive data (cost, margin).
- **Multi-warehouse permissions** if you ever expand — a manager might only have access to their facility.
- **Audit trail** of every action by every user (who scanned what, when, where).
- **Session management** — auto-logout, optional 2FA for admin.

4.9 Admin Dashboard [CORE]

- **Top-line KPIs:** total items, items in storage, items deployed, items in repair, retired this season, capital tied up in inventory.
- **Utilization rate** — what % of inventory was deployed last season; identify dead stock.

- **Revenue per kit** (joining Jobber invoice data) — which client kits are most profitable.
- **Loss / shrinkage report** — items flagged missing.
- **Maintenance backlog** — open repair tickets, aging.
- **Upcoming installs** — next 14 days of Jobber Visits with kit-readiness status.
- **Sync health** — last successful Jobber webhook, queue depth, errors.
- **User activity heatmap** — who's scanning, who's not.
- **Reorder suggestions** — based on consumable depletion rates.

4.10 User Dashboard [CORE]

For app users in the "User" permission role (warehouse staff, ops support, anyone doing scans without admin/manager access). Note: field crew members do **not** need to be users of this app — they don't log in. Inventory tracking happens at the vehicle/warehouse level, not the person level. The User dashboard is for the smaller group of staff who actively operate the app.

- **Today's jobs** — pulled from Jobber Visits scheduled today, with associated kits and pull lists.
- **Vehicle load status** — for each truck going out, what's loaded, what's still missing, what's flagged.
- **Scan-to-load** flow.
- **Open issues** — items flagged from previous jobs needing follow-up.
- **Recently completed** — last 7 days of activity (actions, not "your" actions).
- **Quick photo upload** to active job.

4.11 Reporting & Analytics

Named, printable reports (fixed templates)

- **Pick List report (printable)** — for any selected Jobber Visit (or batch of Visits for a day), generates a clean, printable list of every item that needs to come out of storage and onto a truck. Includes: item name, SKU, QR code, quantity, source bin location, destination kit name, special handling notes. Sortable by bin location (so the warehouse picker walks the most efficient route) or by kit (so items stay grouped). PDF and direct-print formats. One copy per truck/job. This is the most-used physical-paper output of the app during install season.

- **Job Notes report (Jobber-sourced)** — pulls **all internal notes** attached to a given Jobber Job, aggregated across the full lifecycle of that work. Notes get created at different stages and by different people, so the report walks the relationship chain: **Request notes → Quote notes → Job notes → Visit notes → Client notes** (when relevant). Each entry shows: timestamp, author, source object (Quote/Job/Visit/etc.), and the note body, displayed chronologically so you see the full history of decisions, conversations, and field observations for that job in one document. Use case: at install morning huddle, the crew lead prints this for every property they're hitting that day. Also used for end-of-season retrospectives and dispute resolution. Note: Jobber's API exposes notes per-object (`notes: JobNoteUnionConnection` , `notes: QuoteNoteUnionConnection` , etc.) — your app fetches them all and merges in one view.

Standard analytical reports

- **Inventory valuation** (current book value, replacement cost).
- **Deployment history per item** — show me every job this strand has been on.
- **Client kit history** — every season's allocation, condition, repairs for one client.
- **Failure rate by manufacturer/SKU** — which products to stop buying.
- **Vehicle utilization** — how many jobs each truck handled, average load size, miles driven (if odometer captured).
- **Loss and damage report** by season, vehicle, product line.
- **Financial export** — feeds into QuickBooks if connected.
- **Custom report builder** — pick fields, filters, group-bys.
- **Scheduled email reports** — weekly summary to admin.

4.12 Import / Export [CORE — you specified this]

Import

- CSV import for master inventory (initial migration).
- CSV import for customer kit assignments.
- CSV import for bin/location data.
- Excel template downloadable from the app.
- Validation pre-flight: show errors before committing.
- Image bulk import via ZIP (filename = SKU).

- Idempotent re-import (don't duplicate; update by SKU).

Export

- Full inventory CSV / Excel.
- Filtered exports (by category, location, status).
- Audit log export.
- Financial export.
- Backup export (full DB snapshot for disaster recovery).
- API export endpoint for integrations.

4.13 Notifications

- **In-app:** low stock, kit conflict, repair ticket assigned, sync failure.
- **Email:** daily digest, weekly admin summary, urgent alerts.

4.14 Search & Filtering

- Global search across items, kits, clients, jobs.
- Saved filters / smart views.
- Barcode-based instant lookup.
- Type-ahead with fuzzy matching.

4.15 Security & Compliance

- HTTPS-only.
- Encrypted at rest (database-level encryption).
- Role-based access control with least-privilege defaults.
- 2FA for admin (TOTP).
- Session timeout configurable.
- Activity audit log (immutable, exportable).
- Backup strategy (daily automated).
- Data retention policy (especially for departed clients).
- GDPR/CCPA-style data export and deletion endpoints.

4.16 Custom Report Designer [WISH LIST — PHASE 5]

A drag-and-drop tool that lets you build your own reports without code or developer involvement. The Pick List and Job Notes reports above are fixed templates baked into the app; this is the opposite — an open canvas where you decide what fields, filters, groupings, and visualizations you want, save the result as a named report, and re-run it whenever you need it.

Why it belongs in Phase 5, not earlier: Building a custom report designer from scratch is a multi-month project on its own. The pragmatic move is to **embed an existing open-source tool** rather than build one. By Phase 5 you'll have a year of operational data and a clearer sense of what reports you actually need beyond the named ones.

Capabilities you should expect:

- Visual query builder ("notebook" or "step-by-step" UI) that lets you pick a table, filter rows, group by fields, summarize, and visualize — with no SQL required.
- SQL editor for power users who want full control.
- Drag-and-drop dashboard layout — combine multiple charts, KPIs, and tables onto one page.
- 15+ chart types (bar, line, pie, table, map, KPI tile, funnel).
- Saved reports organized in folders with permissions.
- Scheduled email delivery (e.g., "send the weekly inventory summary every Monday at 8am").
- Filters and parameters so one saved report can answer many variants of a question.
- Export to PDF, CSV, Excel.
- AI/natural-language query (newer tools): "show me top 10 clients by revenue this season" → generates the report.

Recommended tools (ranked for your stack):

Tool	Why it fits	Trade-offs
Metabase <i>(primary)</i>	Open source. Easy setup — runs as a single Docker container or JAR. Visual notebook query builder needs no SQL, so you (and managers) can build reports yourself. Native Postgres support means it talks directly to	Embedding into your app is iframe-based, which works for an internal app but feels less "native." Some advanced features

<i>recommendation)</i>	Supabase. Free self-hosted; ~\$85/mo for Cloud Starter if you want managed. Most popular open-source BI tool, 90,000+ companies use it.	(granular permissions, white-labeling) are paid-tier only.
Apache Superset	More powerful — 50+ chart types, deeper SQL IDE, more customization. Free open source.	Significantly harder to set up (Python, Flask, Celery, Redis). Steeper learning curve. Overkill for a single-franchise app unless you grow into multi-warehouse complexity.
Redash	Lightweight, SQL-first. Fast for engineers. Open source.	No drag-and-drop — assumes SQL knowledge. Doesn't match the "intuitive" requirement you set.
Embeddable / Luzmo <i>(commercial)</i>	Truly native embedding (web components, not iframes). Polished UX. Built specifically for embedding inside SaaS apps.	Paid only, starts at meaningful monthly cost. Probably overkill for single-tenant internal use.
Build your own	Full control.	Multi-month build. Don't do this unless none of the above fit.

My pick: Metabase, self-hosted on the same infrastructure as the rest of the app. It's the lowest-effort path to a real drag-and-drop builder, the visual notebook UI matches your "intuitive" requirement, it reads directly from Postgres so no data duplication is needed, and the open-source version is free. You can iframe-embed it inside the app under a "Reports" tab so users don't see it as a separate tool. If you outgrow the free version's permission model later, Metabase Pro is a clean upgrade — same data, same reports, just more controls.

Implementation notes:

- Run Metabase as a separate container (Railway, Fly.io, or a Docker droplet) — about \$5–10/mo.

- Point it at your Supabase Postgres with a read-only DB user (security best practice — no risk of accidental writes from a report).
- Give it a curated set of “models” (named views/tables) so non-technical users see clean field names like “Client Name” and “Bin Location” instead of `clients.name` and `locations.barcode`.
- Use the Metabase embedding token API to single-sign-on from your app to specific dashboards.
- Schedule the Pick List and Job Notes reports as native printable templates (PDF generators in the app itself) — keep those out of Metabase. Metabase is for analytical/exploratory reports; native templates are for consistent, customer-or-crew-facing documents.

5. Database Schema — Suggested Core Tables

users	(id, email, name, role, jobber_user_id, active, ...)
items	(id, sku, name, category_id, manufacturer, serial, barcode, status, condition_grade, purchase_date, purchase_cost, replacement_cost, current_location_id, home_location_id, parent_kit_id, lifecycle_type, photos[], custom_fields_json, retirement_max_seasons, retirement_max_repairs, retirement_condition_floor, seasons_deployed, repair_count, retirement_status)
categories	(id, name, parent_id, default_retirement_max_seasons, default_retirement_max_repairs, default_retirement_condition_floor) -- hierarchical; -- defaults inherit to items unless overridden
locations	(id, name, type [warehouse aisle rack bin vehicle], parent_id, barcode)
kits	(id, name, kit_type [template customer], client_id, property_id, status, home_location_id, photos[])
kit_items	(kit_id, item_id, quantity) -- junction; supports nested
clients	(id, jobber_client_id, name, email, segment [res com mun], ..
properties	(id, jobber_property_id, client_id, address, ...)
jobber_jobs	(id, jobber_job_id, client_id, property_id, status, ...)
jobber_visits	(id, jobber_visit_id, job_id, start_at, end_at, status, ...)
vehicles	(id, name, plate, vin, make, model, year, capacity, location_id, registration_exp, insurance_exp, odometer, status, photos[])
allocations	(id, kit_id OR item_id, jobber_visit_id, allocated_at, checked_out_at, checked_in_at, allocated_by_user_id,

```
        vehicle_id, season_id,
        status [reserved_on_sale|reserved|out|returned|short|missing
replacement_queue (id, item_id, reason, flagged_at, flagged_by_user_id,
        resolution [repair|replace_from_stock|purchase_order],
        po_id, resolved_at, resolved_by_user_id)
maintenance_tickets (id, item_id, opened_at, opened_by, status, description,
        photos[], parts_used_json, labor_minutes, closed_at)
audit_log (id, user_id, action, entity_type, entity_id, before_json,
        after_json, ip, user_agent, occurred_at)
sync_events (id, source [jobber], event_type, payload_json,
        processed_at, status, error)
custom_field_defs (id, applies_to, name, type, options_json)
custom_field_values (entity_type, entity_id, field_def_id, value)
```

6. Tech Stack Recommendations (for Vibe Coding)

For a single-franchise app you'll vibe-code and maintain:

- **Frontend:** Next.js or Remix (React, server-side rendering, easy mobile responsive).
 - **Backend:** Node.js + Express/Fastify, or a meta-framework like Next.js API routes.
 - **Database:** PostgreSQL (relational fits this domain; JSON columns for custom fields).
 - **Auth:** Clerk, Auth0, or NextAuth.js — gives you 2FA out of the box.
 - **File storage:** S3 (or Cloudflare R2 — cheaper).
 - **Background jobs:** BullMQ + Redis, or Inngest, for webhook processing.
 - **Hosting:** Vercel + Supabase, or Railway, or Fly.io. All hobbyist-friendly.
 - **GraphQL client for Jobber:** `graphql-request` or Apollo Client.
 - **Barcode scanning:** `@zxing/browser` (web), or native camera APIs in a wrapper.
 - **PDF / label printing:** `pdfkit` or browser print CSS for bin labels.
-

7. What Competitors Do Well — Specific Examples to Borrow

- **Booqable** — clean online catalog and self-service booking widget; would be the pattern if you ever offer self-quote to residential leads.
- **Rentman** — Gantt-style project planner across multi-day projects with crew + kit on one

view.

- **EZRentOut** — QR code labels and recurring rental rules; mature mobile.
 - **Goodshuffle Pro** — best-in-class real-time inventory calendar with conflict highlighting.
 - **Current RMS** — nested kit assembly with substitution rules; built for AV/lighting rental specifically.
 - **HireHop** — dynamic kit builder with real-time stock; strong for multi-warehouse.
 - **Texada** — drag-and-drop dispatch and asset location maps.
 - **RealGreen** (holiday-lighting-specific) — before/after photos saved to customer profile, clock-in/out for actual labor hours, condition codes for future seasons. Most directly applicable to your industry.
 - **QuotelQ** — built by holiday lighting pros; satellite measurement and self-quoting for homeowners.
-

8. Christmas-Decor-Specific Recommendations

These are the moves I'd make that nobody else in the rental space does well:

1. **Make the customer the primary unit of inventory organization, not the item.** Most rental software is item-first. Yours should be client-first because each client has a near-permanent kit assignment.
2. **"Season" as a first-class entity.** Every allocation, condition record, maintenance ticket, and revenue record should be stamped with a season ID (e.g., "2025–2026 Holiday"). This makes year-over-year reporting trivial.
3. **Pre-build mode in summer.** A dedicated workflow where crews spend Aug–Oct refurbishing and pre-assembling kits, with progress tracking ("142 of 187 client kits ready").
4. **Customer-facing seasonal preview email.** Auto-generate from previous season's kit + photos: "Here's what you had last year — same again? Add anything?" Fed back into Jobber as a Quote draft.
5. **Routing-aware install day plan.** Once you have all the day's properties from Jobber + their kit bin locations, generate the optimal warehouse pull order AND the route order.
6. **Rental vs. purchase distinction in the catalog.** Christmas Decor sells some items outright (permanent year-round lighting per the website) and rents others. Same item

table can flag both, but the lifecycle differs.

7. **Year-over-year condition drift.** Show admins a chart of how each client's kit condition is trending — early warning for clients whose kit is due for a major refresh / upsell.
-

9. Build Phasing Recommendation

If I were vibe-coding this, I'd build in this order so you have something useful fast:

Phase 1 (MVP, 2–4 weeks)

- Auth + 3 roles
- Items, categories, locations, basic CRUD
- CSV import/export
- Jobber OAuth + read-only client/property/job sync
- Admin dashboard with basic counts

Phase 2

- Kits and kit assembly
- Customer ↔ kit linkage
- Allocation / check-out / check-in
- Mobile-friendly scan flow
- User dashboard

Phase 3

- Maintenance tickets and lifecycle
- Reports and analytics — including the **Pick List** and **Job Notes** printable reports
- Two-way Jobber custom-field write-back
- Webhook-driven real-time sync
- Pull-list generation from Jobber Visits

Phase 4

- Photos, condition history, refurb season workflow

- Notifications
- Custom field definitions per object
- Advanced reports / scheduled emails

Phase 5

- Year-over-year auto-renewal of client kits
- Customer-facing seasonal preview
- Optimization (routing, demand forecasting)
- **Custom Report Designer** — embed Metabase against the Postgres DB, expose a curated set of models, iframe into a “Reports” tab in the app

10. Confirmed Architecture Decisions

The following are now locked in based on your answers:

#	Decision	Architectural Implication
1	Single franchise	Single-tenant architecture. Skip the multi-tenant overhead — one DB, one auth realm, one set of config. Faster to build, cheaper to host. If you ever license to other franchisees, refactor later.
2	Single warehouse	No multi-warehouse permission logic, no inter-warehouse transfers, no per-warehouse stock counts. The <code>locations</code> table is still hierarchical (warehouse → aisle → rack → bin → vehicle), but there’s only one root warehouse.
3	Jobber Plus plan	✅ API access confirmed. You can use OAuth, GraphQL, webhooks, and custom field configuration. No blockers.
4	No customer self-service portal	Internal-only app. No public-facing routes, no client login, no online booking widget. Smaller attack surface, simpler auth, no need for client-hub-style features.
5	Mobile-responsive web	Build as a responsive web app, not a native mobile app. Use modern responsive frameworks (Tailwind + Next.js or similar). Camera APIs work fine in mobile browsers for QR scanning. No app store submissions, no native build pipeline.
		Use <code>@zxing/browser</code> or <code>html5-qrcode</code> for in-browser

6	QR codes	scanning. Generate codes server-side with a library like <code>qrcode</code> (Node) and print to standard label sheets (Avery 5160-equivalent or thermal labels via Brother/Dymo/Zebra). Each item, bin, vehicle, and kit gets its own QR.
7	No QuickBooks integration	Skip QBO entirely. Jobber handles your accounting flow — invoices, payments, and reports stay in Jobber. Your app just feeds line items into Jobber Quotes/Jobs and reads back what was billed.
8	Photos live in your app	Build photo storage into the app (S3 or Cloudflare R2). Schema: <code>photos[]</code> JSON column or a dedicated <code>photos</code> table with <code>entity_type</code> , <code>entity_id</code> , <code>url</code> , <code>taken_by_user_id</code> , <code>taken_at</code> , <code>caption</code> , <code>geo_lat</code> , <code>geo_lng</code> . Don't push to Jobber notes — keep them in your DB so you control retention and search. Images compressed/resized on upload.
9	Permanent Lighting + Landscape Lighting in scope	Add to category taxonomy (already done in §4.1 above). Note that these have different lifecycles than seasonal: • Permanent Lighting (year-round outdoor fixtures, often cloud-controlled) is typically <i>sold and installed permanently</i> , not rented. The item record needs a <code>lifecycle_type</code> field [<code>rental_seasonal</code> <code>sold_permanent</code> <code>installed_year_round</code>]. Sold-permanent items leave your inventory at install — track them as “deployed permanently to client X” so you can support warranty/service calls without them counting against available stock. • Landscape Lighting can be either (some clients buy outright, some rent year-round). Same <code>lifecycle_type</code> flag handles it. • Service-call workflow — both product lines generate ongoing service calls (bulb replacement, controller reboots, fixture repairs). Even though the item isn't in your warehouse, the app should still surface its install date, components, and service history when a Jobber Job is created for that property.
10	No compliance/data residency requirements	Skip GDPR/CCPA workflows for v1. Use a US-region database. Standard backups and audit logs are sufficient. Revisit if you ever expand to Canadian Christmas Decor franchises (PIPEDA) or sell to franchisees in other regions.

Now-resolved follow-up decisions

#	Decision	Answer / Rationale
		Vercel + Supabase + Cloudflare R2. This is the cleanest modern

1	Hosting recommendation	stack for a vibe-coded single-tenant app:
• Vercel hosts the Next.js frontend and API routes. Free tier works for early development; ~\$20/mo Pro when you go live.
• Supabase gives you managed Postgres, built-in auth (with 2FA), file storage, real-time subscriptions, and a UI for inspecting data — all in one service. Free tier is generous; \$25/mo Pro for production.
• Cloudflare R2 for photo storage at fraction of S3 cost (\$0.015/GB/month, no egress fees). Alternative: keep photos in Supabase Storage if you want fewer moving parts.
• Background jobs for Jobber webhook processing: use Supabase Edge Functions or a small worker on Vercel/Railway.
Estimated total monthly cost: \$0–\$30 in development, \$50–\$100/mo in production. If you want everything in one place with simpler ops, Railway (\$10–30/mo) is a solid alternative — runs Postgres, web app, and workers under one roof.
2	Season cutover	Active season runs Sept 2 – March 1. Off-season (March 2 – Sept 1) is when refurbishment, kit pre-build, and inventory replacement happen. Schema implication: every season record stores <code>start_date</code> and <code>end_date</code>; the active season is determined by today's date falling within those bounds. Reports default to current season but can be sliced by any prior season. The off-season is a first-class state — the dashboard should look different March 2 onward, surfacing the refurbishment backlog and pre-build progress instead of active deployments.
3	Item retirement threshold	Per-item, with category defaults. Each category (e.g., "C9 Strands", "Wire Frames", "Controllers") has a default retirement profile — max seasons, max repair count, minimum acceptable condition grade. When a new item is added, it inherits its category's defaults, but admins can override at the individual item level for outliers. The system auto-flags items approaching ANY of their thresholds for inspection. Schema reflects this in both <code>categories</code> and <code>items</code> tables (see §5).
4	Crew offboarding	Not applicable. Field crew are not users of this app. Inventory is tracked at the vehicle and warehouse-bin level, not the person level. When a field crew member leaves, no app cleanup is needed. For the small group of warehouse/ops staff who DO have user accounts, standard offboarding applies: deactivate the account, reassign any pending photo uploads or scan tasks to a manager. This is a 5-minute admin action, not a workflow.

End of research document.